EEE3094 Technical Report

Sahas Talasila *230057896*

Contents

Abstract

This is a placeholder for the abstract of the report.

Introduction

Literature Review and Motivation

2.1 Time Series Feature Extraction

Raw time series are high-dimensional, temporally correlated, and often noisy. Although models can ingest sequences directly, feature preprocessing usually improves accuracy and efficiency. The preprocessing choice determines which information is preserved.

Spectral methods project signals into the frequency domain. The fast Fourier transform decomposes signals into sinusoidal components and reveals dominant periodicities. Recent work combines FFT amplitude and phase features with LSTMs to leverage time- and frequency-domain information [FFTLSTM]. Transformer variants using FFT-based preprocessing improve forecasting across sensor datasets by enriching representations with spectral structure [TSFNetFFT]. Wavelet and modal methods provide joint time-frequency localisation. MDCNet uses variational mode decomposition to separate trend and modal components and achieves strong long-horizon performance [MDCNet]. These approaches characterise frequency content or scale-specific energy but do not encode structural relationships between temporal events.

Statistical feature extraction remains in the time domain. Toolkits such as *hctsa* compute large collections of scalar summaries including distributional statistics, autocorrelation measures, and entropy estimates [catch22]. Deep learning models instead learn representations end-to-end from raw sequences. Recurrent and convolutional architectures capture temporal dependencies without manual feature design. THATSN builds a temporal hierarchical tree from multi-cyclic decompositions and aggregates features using a gated LSTM to capture inter-cycle relationships [THATSN]. Despite strong predictive performance, these learned representations are typically opaque.

Across both paradigms, the mapping from raw signal to feature is rarely transparent. Spectral coefficients, statistical summaries, and learned embeddings may support accurate predictions, but they do not provide a clear link between predictions and specific events in the input signal.

2.2 Visibility Graphs for Time Series

Visibility graphs map geometric relationships in time series to graphs: two points connect if no intermediate point blocks their line of sight [NetworkStructureLacasa]. This preserves temporal ordering and reflects underlying dynamics, since degree distributions distinguish periodic, random, and fractal processes [NetworkStructureLacasa]. Extensions include multiplex visibility graphs, where each channel forms a layer and inter-layer edges capture cross-variable dependencies [multiplex_visibility_Lacasa]. Broader overviews cover classification, clustering, and motif discovery [VGforTSBook], while applications to financial and synthetic data demonstrate extraction of short- and long-term structure [VisibilityGraphBasedTSA].

Standard visibility graphs still have structural limitations. They are undirected, contain cycles, and lack a canonical traversal order. Systematic decomposition into interpretable substructures becomes computationally difficult. As a result, they are limited as preprocessing for machine learning. Graph-level statistics such as degree distribution or clustering coefficient can be used as features, but they cannot be linked to specific events in the original signal.

2.3 Path Signatures in Machine Learning

The signature transform, rooted in rough path theory, maps a sequential path to a graded sequence of iterated integrals that uniquely characterises it up to reparametrisation [GeneralisedSignaturesPaper]. Linear functionals on signature terms are dense in the space of continuous functions on paths, providing a universal nonlinear feature map. The output dimensionality depends only on the embedding dimension and truncation depth, not solely on input length, a property well-suited to variable-length sequences.

A generalised signature framework has been developed and benchmarked across 26 multivariate classification tasks, achieving performance comparable to deep recurrent and convolutional architectures [GeneralisedSignaturesPaper]. Applications span handwriting recognition, medical time series, and financial modelling. However, existing work applies signatures to raw or augmented time series. No prior method computes signatures over structurally decomposed paths derived from a graph-based representation of the signal.

2.4 Statistical Feature Selection

The catch22 feature set distils 4,791 time series characteristics from the *hctsa* library down to 22 canonical features via greedy forward selection across 93 classification benchmarks [catch22]. The retained features span distributional, autocorrelation, successive-difference, and fluctuation-scaling categories, achieving comparable classification accuracy to the full library at a fraction of the computational cost ($O(N^{1.16})$). The selection criterion maximises diversity: each feature captures a distinct dynamical property, minimising inter-feature redundancy.

catch22 features are individually named and interpretable in a statistical sense, but they lack structural provenance. A given feature value quantifies *what* property the signal exhibits — it does not identify *where* in the signal or *which* temporal events produced it.

2.5 The Interpretability Gap

The methods surveyed above share a common structural deficiency: the chain of reasoning from raw signal to extracted feature to model prediction is opaque. FFT projects into the frequency domain, discarding temporal locality. Wavelet decomposition retains time-frequency resolution but commits to a fixed basis, and individual coefficients do not correspond to identifiable signal events. catch22 computes named statistical summaries, yet no summary traces to a specific temporal structure. Standard visibility graphs encode pairwise geometric relationships but lack the directed, acyclic structure required for systematic decomposition. Deep learning methods learn representations end-to-end with no human-readable intermediate stage.

Post-hoc explainability tools such as SHAP can identify *which* features drive a prediction. However, the explanatory value of a SHAP attribution depends entirely on the semantic content of the underlying feature. An attribution to a spectral coefficient or an autocorrelation statistic tells the practitioner that a particular frequency or correlation property mattered — it does not reveal which events in the raw signal produced that property.

This work addresses the gap by proposing a preprocessing method in which every extracted feature possesses full structural provenance. The Magnitude-Ordered Visibility Graph transforms a time series into an acyclic hierarchy admitting polynomial-time decomposition into monotone chains. Path signatures computed over these chains yield fixed-dimensional descriptors, each traceable through a specific chain, through specific graph nodes, to specific timestamps and amplitudes in the original signal. When combined with post-hoc attribution, this provenance enables a complete backtrace: from model prediction, through feature importance, through graph structure, to the raw data.

Theoretical Background

It is imperative that the reader understands the limitations of standard visibility graphs and min-max trees. Standard visibility graphs convert a time series into an undirected graph by connecting points that have line-of-sight visibility. Because visibility is symmetric, cycles arise immediately. Any three mutually visible points form a triangle and any clique of size `k` contains multiple cycles. Consequently, no canonical traversal exists, path enumeration between nodes grows exponentially, and cycle detection is required during feature extraction, making large-scale analysis computationally impractical.

IMAGE SHOWING A VISIBILITY GRAPH

A more fundamental limitation is the loss of information inherent to time series. Two natural orderings exist: temporal ordering and magnitude ordering. Standard visibility graphs ignore both and treat all edges as equivalent, regardless of whether they connect dominant peaks or minor fluctuations.

MOVG incorporates both orderings. Magnitude ordering induces a hierarchy in which the global maximum becomes the root and each node selects as parent the nearest visible point with strictly larger amplitude. Temporal ordering preserves directionality so forward and backward traversal differ. These constraints eliminate cycles via monotone value chains: each step moves to a strictly smaller value, preventing return to the starting node. The resulting structure is acyclic and supports polynomial-time decomposition.

This representation remains visually interpretable. The tree mirrors the signal, revealing dominance relationships among peaks and troughs and linking extracted features directly to amplitudes and timestamps in the raw data.

3.1 MOVG Construction

MOVG construction proceeds in two stages: visibility graph construction, then magnitude hierarchy imposition.

3.1.1 Visibility Condition

Given a time series $Y = \{y_0, y_1, \ldots, y_{n-1}\}$, points i and j $(i < j)$ are mutually visible iff every intermediate point k satisfies:

$$y_k \leq y_i + (y_j - y_i) \cdot \frac{k - i}{j - i} \quad \forall\, i < k < j \tag{1}$$

Adjacent points $(j = i + 1)$ are always visible. The visibility graph $G = (V, E)$ adds an edge for each visible pair.

Algorithm 1 Build Visibility Graph

Require: Time series Y of length n

Ensure: Graph $G = (V, E)$

```
function BuildVisibilityGraph(Y)
    G ← graph with V = {0, ..., n − 1}
    for i ← 0 to n − 2 do
        for j ← i + 1 to n − 1 do
            visible ← true
            for k ← i + 1 to j − 1 do
                if Y[k] > Y[i] + (Y[j] − Y[i]) · (k−i)/(j−i) then
                    visible ← false; break
                end if
            end for
            if visible then G.AddEdge(i, j)
            end if
        end for
    end for
    return G
end function
```

Worst-case complexity is $O(n^3)$: $O(n^2)$ pairs, each requiring up to $O(n)$ intermediate checks.

3.1.2 Magnitude Hierarchy

The visibility graph is undirected and cyclic. The magnitude hierarchy converts it into an acyclic tree. The global maximum becomes the root at depth 0. Each remaining node i is assigned:

$$\text{parent}(i) = \arg\min_{c \in C_i} Y[c], \quad C_i = \{j : j < i,\ G.\text{hasEdge}(j, i),\ Y[j] > Y[i]\} \tag{2}$$

Selecting the *smallest* larger visible predecessor produces a tight hierarchy: root-level nodes represent dominant peaks; leaf nodes represent minor fluctuations. Depth directly encodes amplitude significance.

Algorithm 2 Build Magnitude Hierarchy

Require: Time series Y, visibility graph G
Ensure: Root, parent map, level map

function BUILDHIERARCHY(Y, G)
    root $\leftarrow \arg\max_i Y[i]$;   level[root] $\leftarrow 0$
    for $i \leftarrow 0$ **to** $n-1$, $i \neq$ root **do**
        $C \leftarrow \{j : j < i \wedge G.\text{hasEdge}(j, i) \wedge Y[j] > Y[i]\}$
        if $C \neq \emptyset$ **then**
            parent$[i] \leftarrow \arg\min_{c \in C} Y[c]$
            level$[i] \leftarrow$ level[parent$[i]$] $+ 1$
        end if
    end for
    return (root, parent, level)
end function

Hierarchy construction is $O(n^2)$ time, $O(n)$ space. The resulting tree is provably acyclic: any chain through the hierarchy has strictly monotone amplitudes ($v_0 > v_1 > \cdots > v_k$), so returning to v_0 would require $v_k > v_0$, a contradiction.

3.1.3 Worked Example

Consider the 8-sample time series $Y = [3, 1, 4, 2, 5, 1, 3, 2]$, plotted in Figure 1.

Figure 1: MOVG construction on $Y = [3, 1, 4, 2, 5, 1, 3, 2]$. (a) Raw time series. (b) Magnitude hierarchy: node 4 (y=5) is root; parent assignments follow Eq. 2.

Node 4 (y=5) is the global maximum and becomes root. Node 2 (y=4) is its nearest smaller-valued visible predecessor and attaches at depth 1. Nodes 0 and 6 (y=3) attach at depth 2. The remaining nodes populate deeper levels. The tree directly reflects the amplitude hierarchy of the original signal.

3.2 Path Decomposition

Given a selected query node s, the decomposition algorithm enumerates all maximal monotone chains terminating at s via backward traversal through the hierarchy. Two families of chains are recovered: *decreasing chains*, along which amplitudes are strictly monotonically decreasing in forward time ($v_0 > v_1 > \cdots > v_k$), and *increasing chains*, along which they are strictly increasing. These correspond respectively to descents from dominant peaks and ascents toward them.

3.2.1 Density and Chronological Constraints

Two admissibility conditions govern chain validity.

Definition 1 (Chronologically Dense Path)**.** *A path $X = [n_0, n_1, \ldots, n_k]$ is* chronologically dense *if for every consecutive pair (n_i, n_{i+1}), no intermediate node m exists such that (i)* $\min(n_i, n_{i+1}) < m < \max(n_i, n_{i+1})$*, (ii) m is visible from both n_i and n_{i+1}, and (iii) m preserves the monotone trend of the chain.*

The density constraint prevents the omission of structurally relevant intermediate nodes. A second constraint enforces temporal causality: once the traversal moves backward in time, no subsequent forward jump is permitted.

3.2.2 Chain Discovery

Algorithm 3 recovers all maximal decreasing chains via recursive backward search. At each step, the frontier node is extended to all visible predecessors with strictly greater amplitude, subject to density and chronological admissibility. Recursion terminates when no further valid extension exists; chains of length ≥ 2 are retained. Increasing chains are obtained identically with the magnitude inequality reversed.

Algorithm 3 Find Maximal Decreasing Chains

Require: MOVG structure, query node s
Ensure: Set of maximal decreasing chains ending at s

```
function FindDecreasingChains(MOVG, s)
    𝒞 ← ∅
    procedure BackwardSearch(path)
        f ← path[0]                                              ▷ Frontier node
        extended ← false
        for p ∈ predecessors(f) where Y[p] > Y[f] do
            if IsDense([p] ⊕ path) ∧ IsChronological([p] ⊕ path) then
                BackwardSearch([p] ⊕ path)
                extended ← true
            end if
        end for
        if ¬extended ∧ |path| ≥ 2 then
            𝒞 ← 𝒞 ∪ {path}
        end if
    end procedure
    BackwardSearch([s])
    return 𝒞
end function
```

From each maximal chain, all valid subpaths are generated by progressive truncation of the leading element, retaining only those containing s. This yields a comprehensive family of monotone subsequences at varying temporal scales, each anchored at the query node. Complexity is $O(k \cdot n)$ for chain discovery and $O(k \cdot L^2)$ for subpath generation, where k denotes the number of maximal chains and L the mean chain length.

3.2.3 Worked Example (continued)

NEED TO REDO THE EXAMPLES AND DO IT MY WAY

Returning to $Y = [3, 1, 4, 2, 5, 1, 3, 2]$ with query node $s = 7$ (y=2), the decomposition recovers chains by traversing backward through the hierarchy. A decreasing chain might comprise $[4, 6, 7]$ (amplitudes $5 \to 3 \to 2$), capturing the descent from the global maximum through an intermediate peak to the query point. An increasing chain might comprise $[5, 7]$ (amplitudes $1 \to 2$), capturing a local ascent. Longer chains encode coarser-scale trends; shorter subpaths capture finer local structure. Each chain is directly traceable to specific nodes and timestamps in the original signal.

Figure 2: Decomposition at query node s=7 on $Y = [3, 1, 4, 2, 5, 1, 3, 2]$. Decreasing chain $[4, 6, 7]$ (red) captures descent from global maximum. Increasing chain $[5, 7]$ (blue) captures local ascent.

3.3 Path Signatures

Path signatures, derived from rough path theory, provide a method for reducing variable-length sequential data to fixed-dimensional numerically encodable descriptors via iterated integrals. Applied to the chains produced by MOVG decomposition, they yield, what are essentially, compact geometric "fingerprints" that capture trend, curvature, and temporal asymmetry, which are properties no single summary statistic encodes simultaneously.

The decomposition of Section 3.2 yields families of monotone chains. To obtain fixed-dimensional numerical descriptors from these variable-length paths, the signature transform from rough path theory is applied. The signature of a path is a graded sequence of iterated integrals that, under mild regularity conditions, characterises the path up to tree-like equivalence and reparametrisation.

3.3.1 The 1D Degeneracy

For a univariate path $x = (x_1, \dots, x_n) \in \mathbb{R}^1$, the depth-$N$ signature collapses to powers of the net increment:

$$S^{(k)}_{1,\dots,1} = \frac{(\Delta x)^k}{k!}, \quad \Delta x = x_n - x_1 \tag{3}$$

All terms are deterministic functions of a single scalar Δx, rendering higher-order components informationally redundant. This degeneracy motivates embedding the path into a higher-dimensional space prior to signature computation.

3.3.2 2D Embedding and Signature Definition

Each chain $X = [n_0, n_1, \dots, n_k]$ extracted from the MOVG is embedded into $\mathbb{R}^2$ via the time-value map:

$$\gamma : n_i \mapsto (t_i, y_i) = (n_i,\ Y[n_i]) \tag{4}$$

This lifts the path into the time-amplitude plane, where non-trivial signed areas exist and iterated integrals carry geometric meaning. The depth-N signature of the embedded path γ is defined as:

Definition 2 (Depth-N Signature)**.** *Let $\gamma : [t_1, t_n] \to \mathbb{R}^2$ be a piecewise-linear interpolation of the embedded chain. The depth-N signature is the collection of iterated integrals*

$$\mathrm{Sig}_N(\gamma) = \left\{ S^{(i_1,\dots,i_k)} = \int_{t_1 \le u_1 < \dots < u_k \le t_n} d\gamma^{i_1}_{u_1} \cdots d\gamma^{i_k}_{u_k} \;\middle|\; 1 \le k \le N,\ i_j \in \{1, 2\} \right\} \tag{5}$$

For d=2 and N=3, this yields $(2^4 - 1)/(2 - 1) = 15$ components.

3.3.3 Extracted Scalar Projections

Rather than utilising the full 15-term vector, three scalar projections are extracted per chain, each capturing a distinct geometric property of the path:

$$\begin{aligned}
\texttt{sig}[2] &= S^{(2)} = \int dy = y(t_n) - y(t_1) && \text{(net amplitude change)} \\
\texttt{sig}[4] &= S^{(1,2)} = \iint_{u_1<u_2} dt\, dy && \text{(signed area / Lévy area analogue)} \\
\texttt{sig}[8] &= S^{(1,1,2)} = \iiint_{u_1<u_2<u_3} dt\, dt\, dy && \text{(time-weighted acceleration)}
\end{aligned} \tag{6}$$

The first-order term $\texttt{sig}[2]$ quantifies overall displacement. The second-order term $\texttt{sig}[4]$ measures the signed area enclosed between the path and the chord connecting its endpoints — distinguishing concave from convex trajectories with identical net change. The third-order term $\texttt{sig}[8]$ captures temporal asymmetry: whether amplitude changes are concentrated early or late in the path. Together, these three projections provide a compact yet discriminative characterisation of each chain's geometry.

3.3.4 Aggregate and Expanded Signatures

Two modes of signature aggregation are employed across the chain families produced by decomposition:

Definition 3 (Aggregate Signature)**.** *For a set of m chains $\{X_1, \ldots, X_m\}$ decomposed at query node s, the* aggregate signature *computes the mean of each scalar projection across all chains:*

$$\bar{\sigma}_k = \frac{1}{m} \sum_{j=1}^{m} \texttt{sig}[k]_j, \quad k \in \{2, 4, 8\} \tag{7}$$

This yields a single 3-dimensional descriptor summarising the typical chain geometry at s.

Definition 4 (Expanded Signature)**.** *The* expanded signature *retains all 15 components of the depth-3 signature for each chain, concatenated with distributional statistics (variance, skewness, kurtosis) across chains. This provides a higher-dimensional but more expressive representation.*

The aggregate form is employed for computational efficiency and downstream interpretability; the expanded form is reserved for settings where richer characterisation is required.

3.3.5 Worked Example (continued)

Returning to the decreasing chain $[4, 6, 7]$ from $Y = [3, 1, 4, 2, 5, 1, 3, 2]$, the 2D embedding produces $\gamma = [(4, 5), (6, 3), (7, 2)]$. The signature projections are:

- $\texttt{sig}[2] = 2 - 5 = -3$ (net descent of 3 units)
- $\texttt{sig}[4] < 0$ (path bows below the direct chord, indicating concavity)

- `sig`[8]: sign and magnitude encode whether the descent accelerates or decelerates over time

These scalar values become features in the downstream machine learning pipeline. Crucially, each is traceable through the chain to specific nodes and timestamps in the original signal. This is the provenance that underpins the interpretability claim of the MOVG framework.

3.4 Baseline Preprocessing Methods

Three established feature extraction methods are employed as comparative baselines. Each is summarised below with attention to its representational assumptions and interpretability properties.

3.4.1 Fast Fourier Transform (FFT)

The discrete Fourier transform decomposes a length-N signal into N complex-valued spectral coefficients:

$$X_k = \sum_{n=0}^{N-1} x_n \, e^{-2\pi i k n/N}, \quad k = 0, 1, \ldots, N-1 \tag{8}$$

Each coefficient X_k encodes the amplitude $|X_k|$ and phase $\arg(X_k)$ of a sinusoidal component at frequency k/N. The fast Fourier transform computes Eq. 8 in $O(N \log N)$ time. As a feature extraction method, the amplitude spectrum is typically retained while phase information is discarded. This projection into the frequency domain identifies dominant periodicities but sacrifices temporal locality: the transform integrates over the entire window, providing no information about *when* a given frequency component is active. Individual spectral coefficients lack direct structural provenance in the time domain.

3.4.2 Wavelet Decomposition

The discrete wavelet transform addresses the temporal locality limitation of FFT by decomposing a signal into coefficients localised in both time and scale. A mother wavelet $\psi(t)$ is scaled and translated to produce a family of basis functions:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \, \psi\left(\frac{t-b}{a}\right) \tag{9}$$

where $a > 0$ controls scale (inversely related to frequency) and b controls temporal position. Multiresolution analysis decomposes the signal into approximation and detail coefficients at successive dyadic scales, yielding a time-frequency representation. This provides finer temporal resolution at high frequencies and finer frequency resolution at low frequencies. However, the choice of mother wavelet and decomposition depth constitutes a *basis commitment*: the representation is optimal only for signals whose morphology matches the chosen wavelet family. The resulting coefficients are not directly traceable to named structural properties of the signal.

3.4.3 catch22

The catch22 feature set comprises 22 canonical time series characteristics selected from an initial library of 4,791 features via a greedy forward selection procedure optimised for classification performance across 93 benchmark datasets. The features span distributional statistics (histogram modes, outlier measures), temporal autocorrelation (first minimum of autocorrelation, automutual information), successive differences (proportion exceeding a threshold, longest stretch of consecutive increases), and fluctuation scaling (detrended fluctuation analysis exponents). The set computes in $O(N^{1.16})$ and is designed to be maximally diverse: each feature captures a distinct dynamical property.

From an interpretability standpoint, catch22 features are domain-agnostic statistical summaries. While individually computable and named, they lack structural provenance: a given feature value cannot be traced to specific temporal events or structural relationships within the signal. This contrasts with MOVG features, where each extracted quantity maps to identifiable nodes, chains, and hierarchy levels in the constructed graph, enabling a complete backtrace from prediction to raw signal.

4.1 Project Deliverables

This project produced three deliverables. First, a formal algorithmic specification of the MOVG framework, comprising definitions, proofs, and pseudocode. This was accompanied by an installable Python package (13 modules, `pyproject.toml`, 45 unit tests). Second, API documentation for package usage for future research, in the RESTful API style (add references). Third, a preprocessing benchmark evaluating MOVG features against three established baselines across seven regression models, together with a post-inference interpretability chain enabling backtrace from model predictions to raw signal events, as well.

4.2 Feature Extraction Pipeline

The MOVG framework extracts 167 features per time series window, organised across 14 categories: path signatures (indices 0–14 plus log-signature norm), tree structure (depth, branching factor, number of roots/leaves/internals), node amplitude statistics at each hierarchy level, level-wise distributional summaries, degree distribution moments, temporal spacing statistics, edge counts, subtree sizes, graph-theoretic measures (diameter, density, clustering coefficient), spectral properties (algebraic connectivity, spectral radius), and standard visibility graph comparison metrics.

4.2.1 Feature Integrity

Of the 167 features, 12 belong to a raw window statistics category. These are computed directly from the signal window and are not derived from the MOVG graph. They were retained and reported separately to quantify their contribution relative to the 155 structural features. This separation prevents conflating graph-derived predictive power with simple statistical baselines, since early experiments showed that `win_*` features created a near-trivial mapping for linear models during flat signal phases, inflating GP and Ridge performance.

The raw window features are:

- `win_mean`, `win_std`, `win_rms`
- `win_min`, `win_max`, `win_range`
- `win_skew`, `win_kurt`, `win_peak_to_peak`
- `win_crest_factor`, `win_n_local_max`, `win_n_local_min`

4.2.2 Dimensionality Reduction

The 155 structural features were reduced to 32 via a three-stage pipeline. First, features with absolute Pearson correlation below 0.30 with the target variable were discarded. Second, iterative variance inflation factor (VIF) screening removed features with VIF exceeding 5, eliminating multicollinearity. Third, domain-driven curation retained the most interpretable representatives from each MOVG category (e.g., preferring tree depth over redundant branching statistics). The resulting 32-feature set spans signatures, hierarchy depth, root/leaf amplitude statistics, and degree distribution moments.

4.2.3 Baseline Feature Sets

Three baseline feature sets were computed on identical windows for fair comparison: catch22 (22 statistical features), FFT (amplitude spectrum coefficients), and discrete wavelet transform (detail and approximation coefficients at dyadic scales). All feature sets share the same train/test splits and evaluation protocol.

4.3 Models and Evaluation

Seven regression models were employed, spanning four learning paradigms: tree-based (XGBoost, LightGBM, Random Forest), kernel-based (SVR, Gaussian Process), linear (Ridge Regression), and instance-based (K-Nearest Neighbours). This diversity tests whether MOVG features benefit models uniformly or proportionally to their inability to discover structure independently from raw inputs.

SVR was initially tuned with C=0.1, which produced near-constant predictions across all feature sets. The regularisation penalty dominated the loss, causing the model to ignore input features entirely. Re-tuning with higher C values recovered meaningful predictions. Both configurations are reported to demonstrate the sensitivity of kernel methods to hyperparameter selection when operating on structural features.

Evaluation followed an expanding-window walk-forward cross-validation protocol. The training set grows with each split while the test set advances forward in time, preventing data leakage. This design preserves temporal ordering and reflects realistic deployment conditions in which a model is periodically retrained as new data arrives. The expanding (rather than sliding) window was chosen because historical observations remain informative: discarding early data would sacrifice context that may improve generalisation. All four feature sets (MOVG-32, catch22, FFT, wavelet) share identical splits, targets, and evaluation metrics (RMSE, MAE) to ensure fair comparison.

4.4 Post-Inference Backtrace

The central contribution of this work is not predictive performance but the ability to explain *why* a model produced a given output. The backtrace procedure establishes a three-layer chain of provenance from a model prediction to identifiable events in the raw signal.

At the first layer, SHAP (SHapley Additive exPlanations) attributes the prediction to individual MOVG features, yielding a ranked list of contributions per feature. At the second layer, each high-attribution feature is mapped to the MOVG graph structure that produced it — a specific chain, a set of nodes, or a hierarchy level. At the third layer, those graph elements are projected back onto the original time series via their stored time indices and amplitude values, identifying the precise temporal events that drove the prediction.

Algorithm 4 formalises this procedure.

Algorithm 4 Post-Inference Backtrace

Require: Trained model $\mathcal{M}$, input feature vector $\mathbf{x}$, MOVG structure $\mathcal{G}$, raw signal Y
Ensure: Highlighted timestamps and amplitudes in Y

```
function BACKTRACE(M, x, G, Y)
    ŷ ← M.predict(x)
    φ ← SHAP(M, x)                         ▷ Per-feature attributions
    F_k ← top-k features by |φ_i|
    for f ∈ F_k do
        N_f ← MapToNodes(G, f)             ▷ Graph nodes producing f
        T_f ← {(t_n, Y[t_n]) : n ∈ N_f}    ▷ Timestamps + amplitudes
    end for
    return {(f, φ_f, T_f) : f ∈ F_k}
end function
```

The output is a set of annotated signal regions: for each influential feature, the practitioner sees which raw data points contributed to that feature's value and, via SHAP, how much that feature shifted the prediction. This contrasts with SHAP applied to spectral or statistical features, where an attribution to (for example) a Fourier coefficient or an autocorrelation statistic identifies *what type* of property mattered but not *which specific temporal events* produced it.

Figure 3: Post-inference backtrace. Layer 1: SHAP attributes prediction to ranked features. Layer 2: each feature maps to specific MOVG nodes and chains. Layer 3: graph elements project onto the raw signal, identifying the temporal events driving the prediction.

Results

Discussion

Conclusion

Constraints and Future Work

References

Appendix